

```

struct tcphdr {
    __be16 source;
    __be16 dest;
    __be32 seq;
    __be32 ack_seq;
    #if defined(__LITTLE_ENDIAN_BITFIELD)
        __u16 res1:4,
            doff:4,
            fin:1,
            syn:1,
            rst:1,
            psh:1,
            ack:1,
            urg:1,
            ece:1,
            cwr:1;
    #elif defined(__BIG_ENDIAN_BITFIELD)
        __u16 doff:4,
            res1:4,
            cwr:1,
            ece:1,
            urg:1,
            ack:1,
            psh:1,
            rst:1,
            syn:1,
            fin:1;
    #else
        #error "Adjust your <asm/byteorder.h> defines"
    #endif
    __be16 window;
    __sum16 check;
    __be16 urg_ptr;
};

```

Figure 7: Structure of the TCP header

```

struct udphdr {
    __be16 source;
    __be16 dest;
    __be16 len;
    __sum16 check;
};

fprintf(log_txt, "\n");
t->Source Port      : %d\n", ntohs(udp->source));
fprintf(log_txt, "\n");
t->Destination Port : %d\n", ntohs(udp->dest));
fprintf(log_txt, "\n");
t->UDP Length       : %d\n", ntohs(udp->len));
fprintf(log_txt, "\n");
t->UDP Checksum     : %d\n", ntohs(udp->check));

```

Figure 8: Structure of the UDP header

Similarly, we can access the TCP header field.

Extracting data

After the transport layer header, there is data payload remaining. For this, we will move the pointer to the data, and then print.

```
unsigned char * data = (buffer + iphdrlen + sizeof(struct ethhdr) + sizeof(struct udphdr));
```

Now, let's print data, and for better representation, let us print 16 bytes in a line.

```

*****UDP Packet*****
Ethernet Header
  |Source Address      : 9C-2A-70-08-50-ED
  |Destination Address : 00-07-E5-12-0F-8F
  |Protocol           : 8

IP Header
  |Version      : 4
  |Internet Header Length : 5 DWORDS or 20 Bytes
  |Type Of Service : 10
  |Total Length   : 33 Bytes
  |Identification : 10281
  |Time To Live   : 64
  |Protocol       : 17
  |Header Checksum : 19134
  |Source IP      : 10.240.253.53
  |Destination IP : 255.255.255.255

UDP Header
  |Source Port      : 23451
  |Destination Port : 23452
  |UDP Length       : 13
  |UDP Checksum     : 0

Data
AA BB CC DD EE 00 00 00 00 00 00 00 00 00 00 00 00

```

Figure 9: UDP packet

```

*****TCP Packet*****
Ethernet Header
  |Source Address      : 9C-2A-70-08-50-ED
  |Destination Address : 00-07-E5-12-0F-8F
  |Protocol           : 8

IP Header
  |Version      : 4
  |Internet Header Length : 5 DWORDS or 20 Bytes
  |Type Of Service : 0
  |Total Length   : 60 Bytes
  |Identification : 21505
  |Time To Live   : 64
  |Protocol       : 6
  |Header Checksum : 29540
  |Source IP      : 10.240.253.53
  |Destination IP : 208.80.154.224

TCP Header
  |Source Port      : 38555
  |Destination Port : 80
  |Sequence Number  : 4210678548
  |Acknowledge Number : 0
  |Header Length    : 10 DWORDS or 40 BYTES
  |-----Flags-----
    |Urgent Flag      : 0
    |Acknowledgement Flag : 0
    |Push Flag       : 0
    |Reset Flag      : 0
    |Synchronise Flag : 1
    |Finish Flag     : 0
  |Window size      : 14600
  |Checksum         : 29573
  |Urgent Pointer    : 0

Data
00 00 00 00 00 A0 02 39 08 73 85 00 00 02 04 05 B4
04 02 08 0A 00 00 8F 15 00 00 00 00 01 03 03 00

```

Figure 10: TCP packet

```

int remaining_data = buflen - (iphdrlen + sizeof(struct ethhdr)
+ sizeof(struct udphdr));

for(i=0;i<remaining_data;i++)
{
    if(i%0 && i%16==0)
        fprintf(log_txt, "\n");
    fprintf(log_txt, " %2X ", data[i]);
}

```